

- This mutex will only be released when `robot_B` has arrived at `storeroom_queue_0`, making space for the next robot to move to `storeroom_queue_1`.

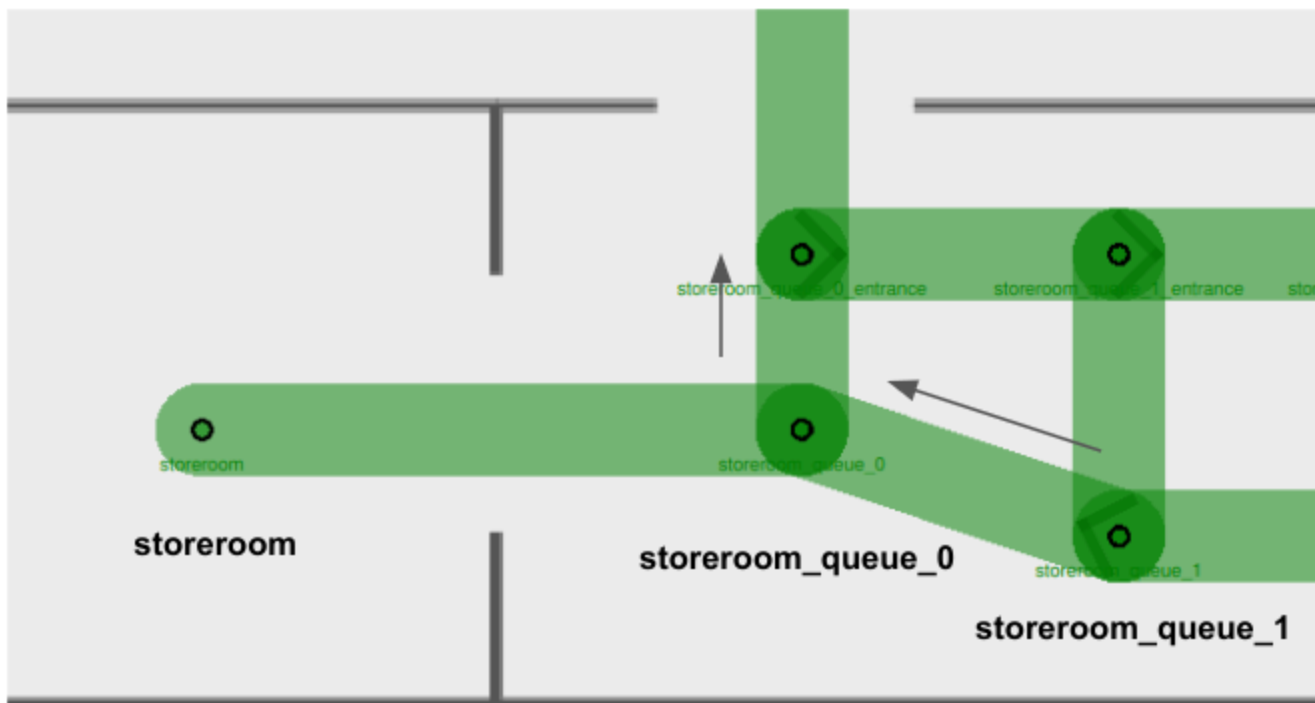
Lane Closure

RMF supports lane closure at runtime. When a `LaneRequest` is published, RMF will update the navigation graph with any lane openings or closures. No robots will be commanded to travel on a closed lane until it is marked as open again.

While this feature is primarily useful for site maintenance and other situations where an area should be temporarily off-limits to robots, it also allows for custom conditional lane traversal.

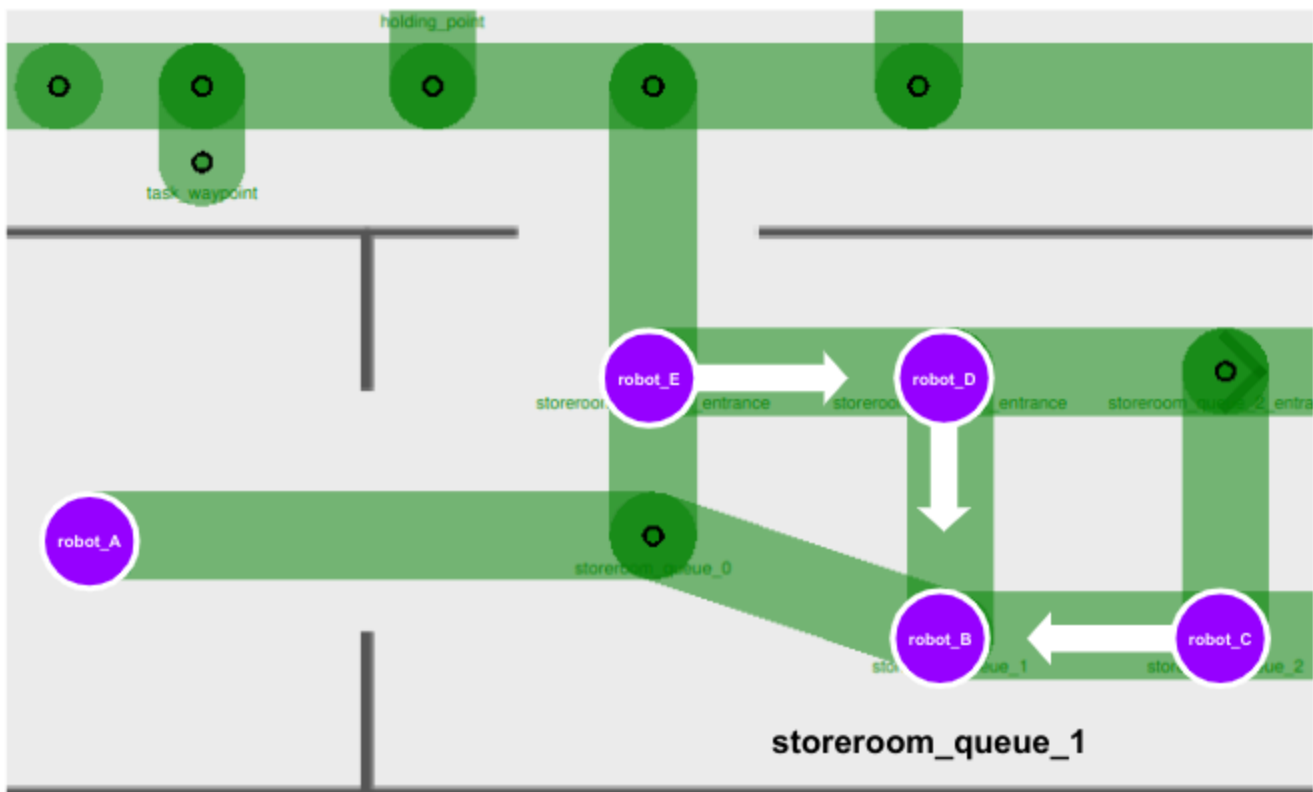
Example 1: Maximum room occupancy

Lane closures can be useful to optimize queue systems. Consider the above example where robots are queueing to enter a storeroom.



Suppose a `robot_A` is currently inside the storeroom, and `robot_B`, `robot_C`, `robot_D` and `robot_E` are all tasked to enter the same room. If we do not define a queueing system for our

robots, it is possible for them to come in from any origin and clutter the storeroom entrance like this:



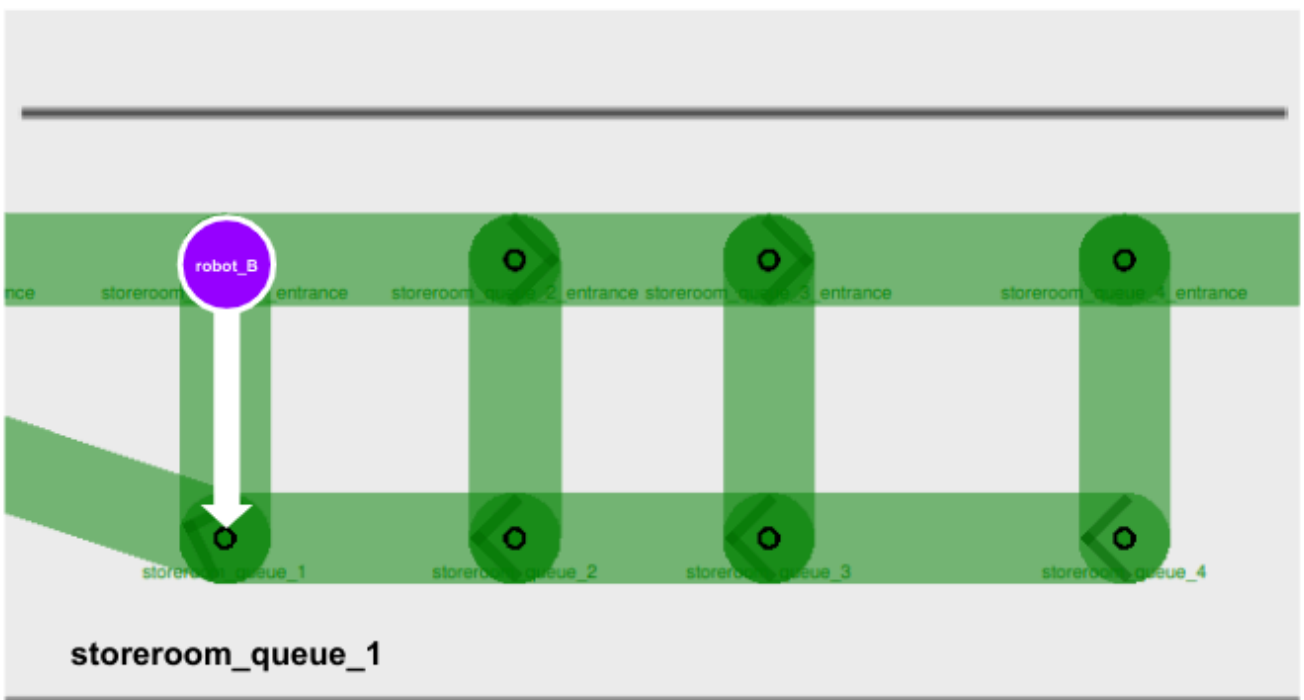
This is undesirable as there is no queue or sequence formed for waiting robots to enter the storeroom systematically. Furthermore, there is a possibility of waiting robots to block the storeroom exit, e.g. `robot_E`, creating a deadlock in this area.

At the same time, we also wouldn't want to remove the bridging lanes entirely, as it would force our robots to take unnecessary and possibly long detours in order to enter the queue, as illustrated below.

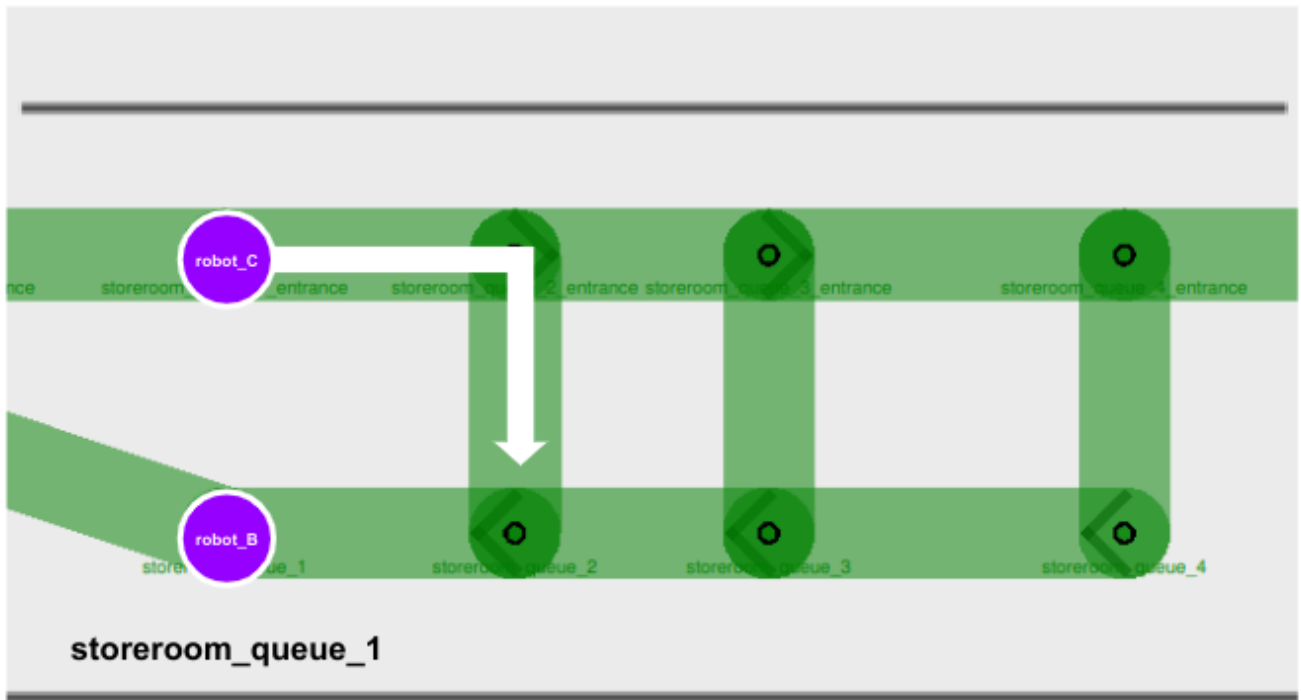


In this case, we can implement conditional lane closures on these bridging lanes.

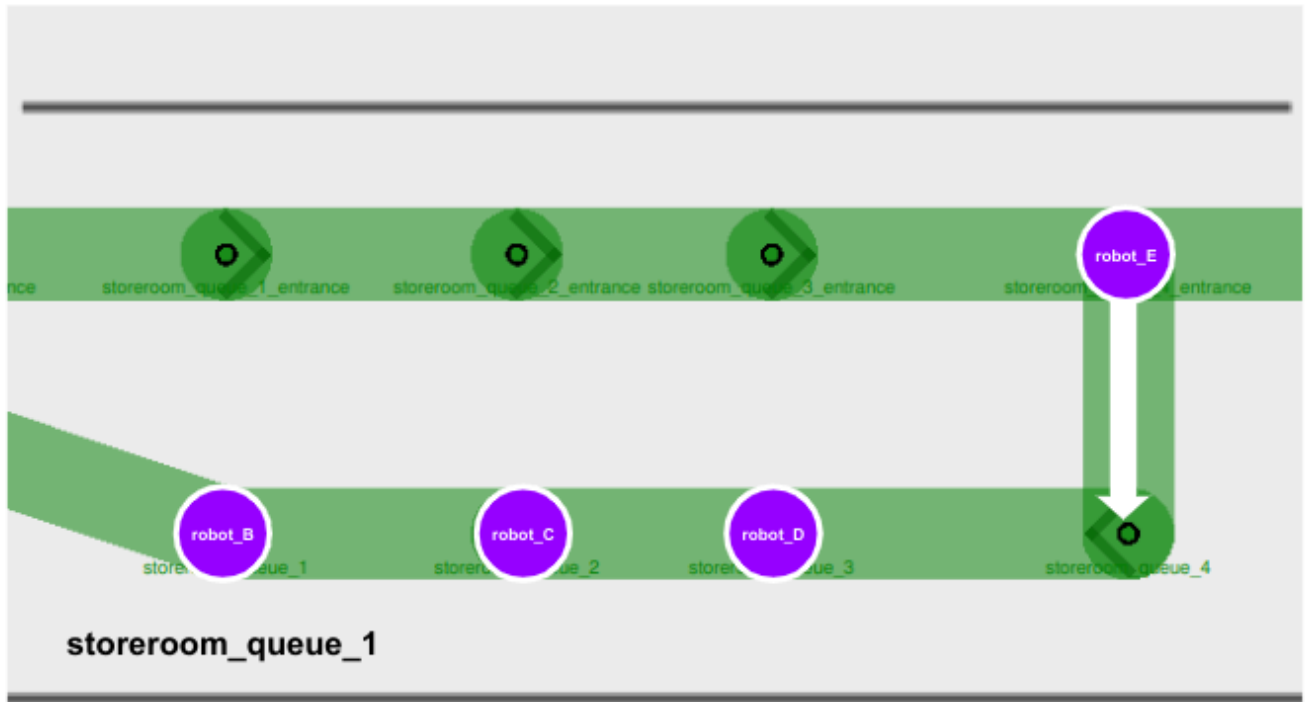
- When `robot_B` arrives outside the storeroom with an empty queue, it can move directly to `storeroom_queue_1` and wait there.



- In your fleet adapter or a separate ROS 2 node, you can add a function to detect waypoint occupancy. If the waypoint `storeroom_queue_1` is occupied, publish a `LaneRequest` to close the waypoint's bridging lane. Any other robots that come along will be restricted to join the queue on the next waypoint.



- As more robots join the queue, we close off more bridging lanes if they lead to a queue waypoint that is already occupied.



Example 2: Taking alternate lifts

Lane closures are also helpful when you need to forcefully re-route robots regardless of what RMF determines to be the most optimized path. Consider this example:

- Your site consists of two lifts, `lift_A` and `lift_B`, that are far apart from each other.
- You have two robots, `robot_A` and `robot_B`, both parked right beside `lift_A` on Level 1.
- Each robot is allocated a task to travel to Level 2, which is accessible via both `lift_A` and `lift_B`.
- When `lift_A` is in use by `robot_A`, RMF might command `robot_B` to queue for the same `lift_A` because it is calculated to be the most optimized path, since `lift_B` is considerably far away.
- However, your site is subjected to several external environmental factors, e.g. foot traffic, that makes it suboptimal for robots to queue for an occupied lift.
- In this case, you may opt to implement a conditional lane closure where the entrance lane to `lift_A` closes as long as there is a robot occupying the lift. This would prompt RMF to re-route `robot_B` to use `lift_B` instead.

The use of lane closures is flexible and extensible to many custom logics. You may consider utilizing them wherever appropriate.

Speed Limits

You are able to create priority in the paths taken by fleet robots by assigning speed limits to lanes. The speed limit property is originally designed to dictate robot velocities in specific areas. However, if it is not necessary for you to relay velocity data to your integrated robots, you can use this feature to create "preferred" lanes in your navigation graph instead.

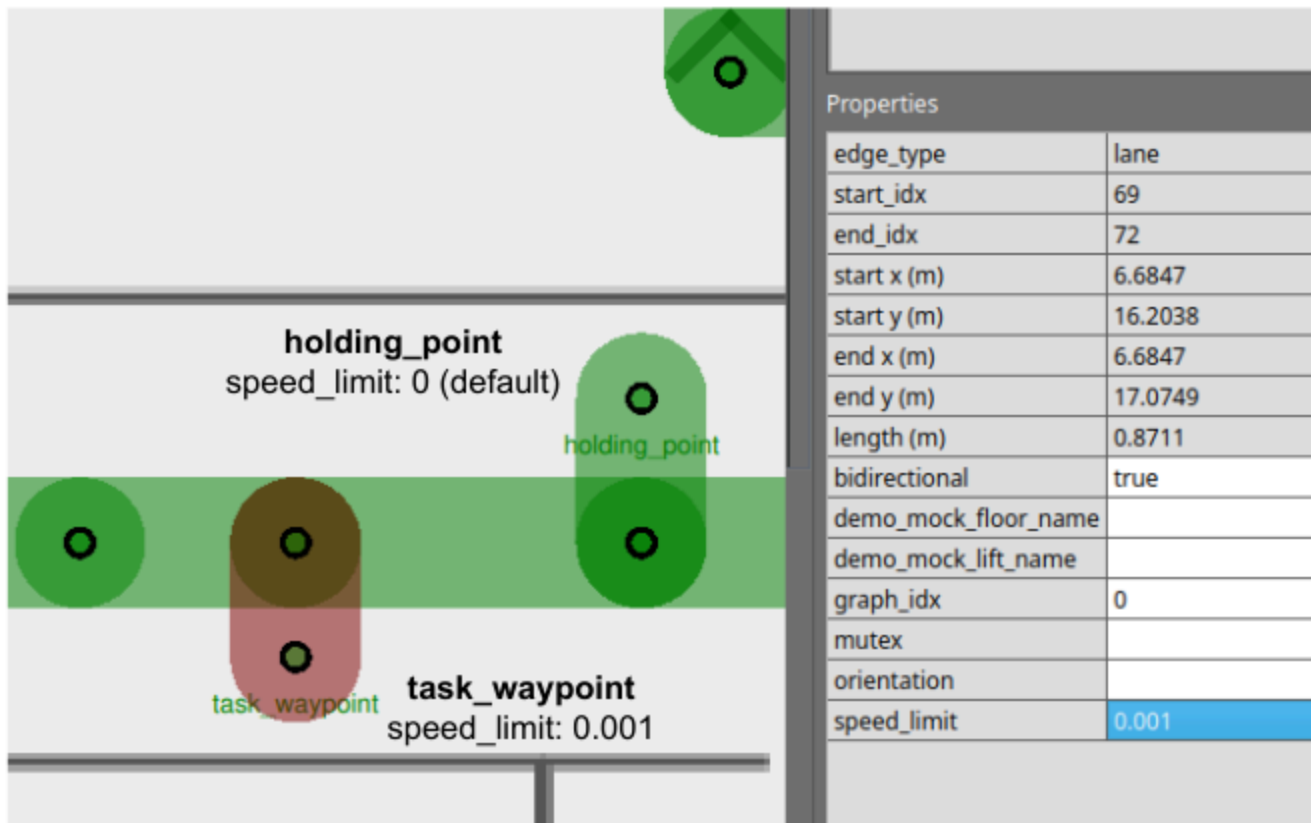
RMF translates lanes with lower speed limits to higher cost, since theoretically it would take a longer time for a robot to travel across that lane. Hence RMF would usually direct robots to take routes involving lanes with higher or no speed limits if an alternate path exists. By default, lanes do not have any speed limits, indicated with `0.0` under the speed limit property.

With this, you may assign low speed limits to lanes that are necessary for certain tasks, but otherwise should be avoided by robots.

Example: Task waypoints vs. Holding points

Holding points (leaf nodes) are extremely useful and important in long, narrow hallways with a single bidirectional lane. They allow robots travelling in opposite directions to give way to one another. However, in certain sites these leaf nodes are not meant to be holding points, but instead are waypoints where tasks are being performed. This means no robots should be allowed on these waypoints unless they are assigned to perform a task there, even during traffic negotiation.

To prevent robots travelling across long hallways from using task waypoints as holding points, you can assign a low speed limit (e.g. `0.001`) to lanes connected to these waypoints.



Combining Strategies

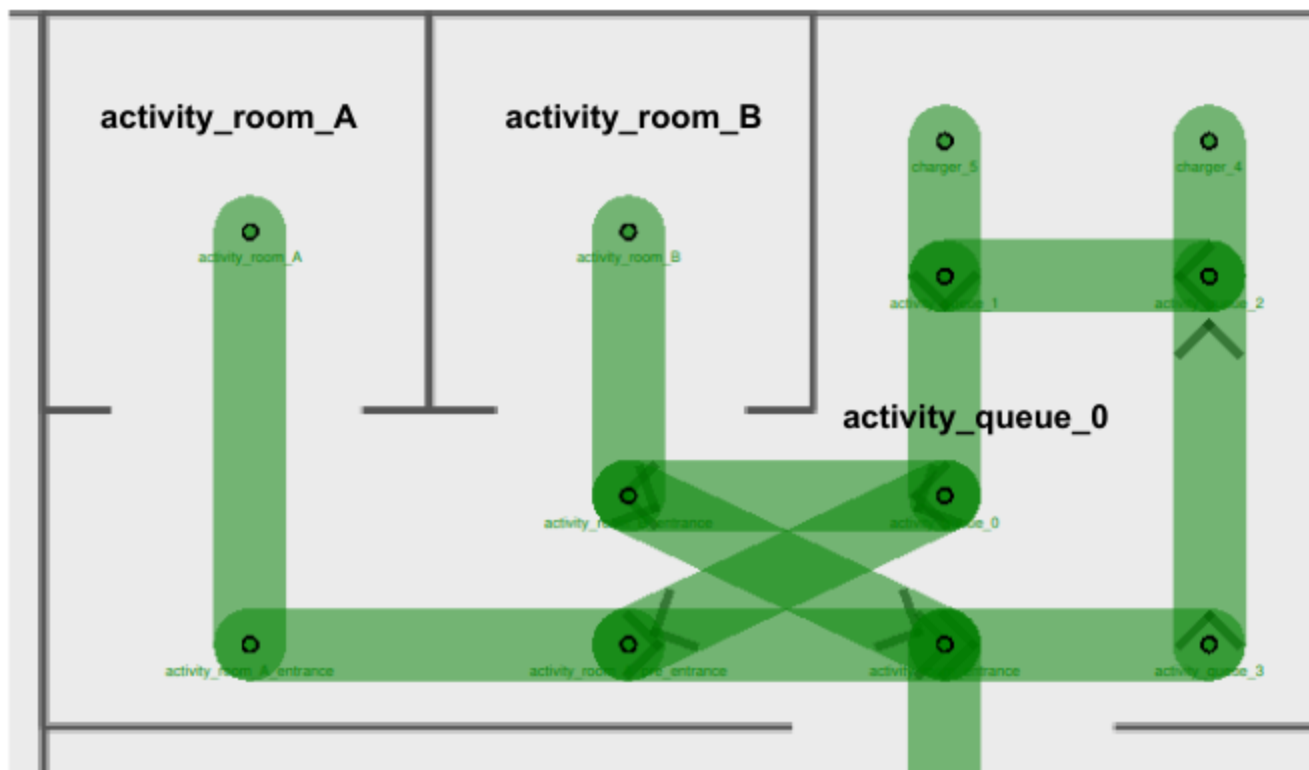
Combining the use of multiple features and strategies can help us navigate through more complicated scenarios.

Example: Maximum occupancy in two separate rooms

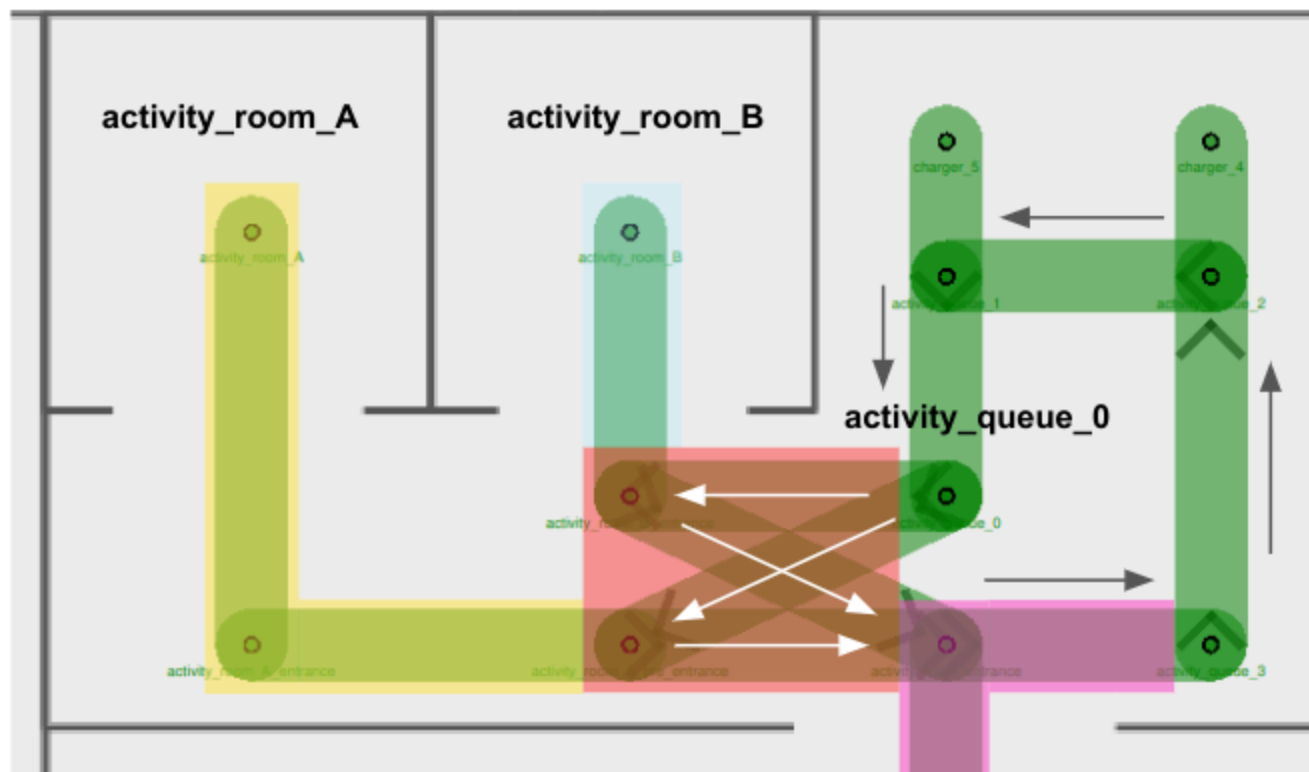
Consider this example:

- There are two activity rooms, `activity_room_A` and `activity_room_B`, each with a maximum occupancy of one robot.
- You would like robots waiting to enter either room to queue up outside, and enter only when their target room becomes vacant.

A navigation graph design to handle this scenario could look like this:



With detailed illustrations of the mutex groups and unidirectional lanes assigned:



In this navigation graph,

- There are a few mutex groups present:
 - `activity_room_A_mutex` in yellow
 - `activity_room_B_mutex` in blue
 - `activity_cross_section_mutex` in red
 - `activity_entrance_mutex` in pink
- Arrows represent the direction assigned for unidirectional lanes
- Unidirectional lanes on the right of the diagram form a queue to enter either room
- Four unidirectional lanes intersect one another right outside `activity_room_B`, forming a cross-section for entrance into/exit from the activity rooms
 - Using separate unidirectional lanes for each activity room enables the next robot to enter its target room as long as it is vacant. Otherwise, it would continue to wait outside when any room is occupied, even if its target room is empty.
- *(Not shown in diagram)* Lane closure logic is implemented for entrance lanes inside the cross-section area. Details are elaborated in [Scenario 3](#) below.

This setup introduces several different possible scenarios. Let's take a look at some of the more problematic ones and how our graph strategy deals with them.

Scenario 1: Two robots exiting activity rooms at the same time

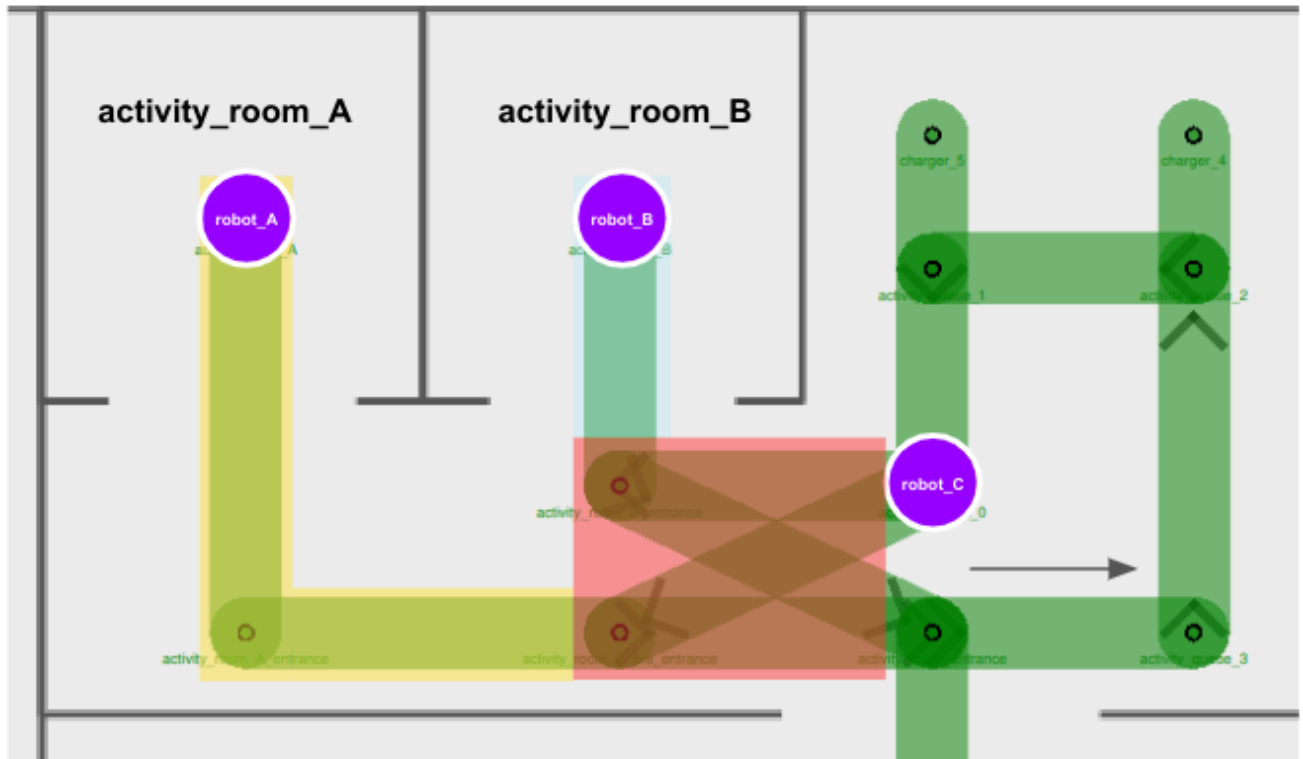
- Suppose a `robot_A` currently occupies `activity_room_A` and a `robot_B` occupies `activity_room_B`. Both robots complete their tasks in the respective rooms and are ready to exit.
- `robot_A` holds the `activity_room_A_mutex` (in yellow), and `robot_B` holds `activity_room_B_mutex` (in blue).
- In order to exit, the robots would need to acquire the `activity_cross_section_mutex` (in red). Depending on which robot obtains the mutex in red first, they would be allowed to travel on their respective exit lane to leave the area. Additionally, they would also need to obtain the `activity_entrance_mutex` (in pink), since the exit waypoint belongs to this mutex group.
- The assignment of mutex groups ensures that only one robot is travelling within the cross-section area at any time, preventing any potential crashes or conflicts.

Scenario 2: Two robots waiting to use cross-section lanes

- Suppose a `robot_A` currently sits on `activity_queue_0`, ready to enter `activity_room_A` which is currently empty. Another `robot_B` is inside `activity_room_B`, ready to exit.
- Similar to above, the `activity_cross_section_mutex` (in red) ensures that `robot_A` and `robot_B` will not travel on any of the cross-section lanes at the same time.

Scenario 3: Robot queueing for an occupied room

- Suppose a `robot_A` currently occupies `activity_room_A` and a `robot_B` occupies `activity_room_B`. Both robots complete their tasks in the respective rooms and are ready to exit.
- A third `robot_C` is sitting on `activity_queue_0`, waiting to enter `activity_room_B`.



- `robot_A` holds the `activity_room_A_mutex` (in yellow), and `robot_B` holds `activity_room_B_mutex` (in blue).
- In order for any robot to move along to their next destination, they would need to obtain the `activity_cross_section_mutex` (in red). From the scenarios above, we know that the red mutex helps to facilitate a turn-by-turn use of the cross-section lanes.
- However, if `robot_C` happens to obtain the red mutex *first*, all three robots would come into a deadlock, since
 - `robot_A` and `robot_B` wouldn't be able to obtain the red mutex until `robot_C` releases it, but
 - `robot_C` wouldn't be able to obtain the blue mutex in order to enter `activity_room_B`, until `robot_B` releases it.